2303A51945
P. Naga Shiva Chaitanya

1) Binary Search :

```java
class Solution {
    public int search(int[] nums, int target) {
        int l = 0;
        int h = nums.length - 1;
        while(l <= h) {
            int m = l + (h - l) / 2;
            if(nums[m] == target) return m;
            else if(nums[m] > target) h = m - 1;
            else l = m + 1;
        }
        return -1;
    }
}
```

2) Search Insert Position :

```java
class Solution {
    public int searchInsert(int[] nums, int target) {
        int low = 0;
        int high = nums.length - 1;
        int ans = nums.length;
        while(low <= high) {
            int mid = low + (high - low) / 2;
            if(nums[mid] >= target) {
                ans = mid;
                high = mid - 1;
            }
            else low = mid + 1;
        }
        return ans;
    }
}
```

3) First Bad Version :

```java
/* The isBadVersion API is defined in the parent class VersionControl.
      boolean isBadVersion(int version); */

public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int l = 0;
        int h = n;
        while(l <= h) {
```

```
            int mid = l + (h - l) / 2;
            if(isBadVersion(mid)){
                h = mid - 1;
            }
            else l = mid + 1;
        }
        return l;
    }
}
```

4) Sqrt(x) :

```
class Solution {
    public int mySqrt(int x) {
        if(x < 2) return x;

        int l = 2; int h = x/2;
        while(l <= h) {
            int m = l + (h - l) / 2;
            if(m == x / m) return m;
            else if(m < x / m ) {
                l = m + 1;
            }
            else h = m - 1;
        }
        return h;
    }
}
```

5) Find First and Last Position of Element in Sorted Array :

```
class Solution {
    public int[] searchRange(int[] nums, int target) {
        int n = nums.length;
        int first = -1;
        int low = 0, high = n - 1;
        while(low <= high) {
            int mid = low + (high - low) / 2;
            if(nums[mid] == target) {
                first = mid;
                high = mid - 1;
            }
            else if(nums[mid] > target) high = mid - 1;
            else low = mid + 1;
        }
        int last = -1;
        low = 0; high = n - 1;
```

```
        while(low <= high) {
            int mid = low + (high - low) / 2;
            if(nums[mid] == target) {
                last = mid;
                low = mid + 1;
            }
            else if(nums[mid] > target) high = mid - 1;
            else low = mid + 1;
        }
        return new int[] {first, last};
    }
}
```

6) Search in Rotated Sorted Array :

```
class Solution {
    public int search(int[] nums, int target) {
        int n = nums.length;
        int low = 0;
        int high = n - 1;
        while(low <= high) {
            int mid = low + (high - low) / 2;
            if(nums[mid] == target) return mid;
            if(nums[low] <= nums[mid]) {
                if(nums[low] <= target && target <= nums[mid]){
                    high = mid - 1;
                }
                else low = mid + 1;
            }
            else{
                if(nums[mid] < target && target <= nums[high]){
                    low = mid + 1;
                }
                else high = mid - 1;
            }
        }
        return -1;
    }
}
```

7) Find Peak Element :

```
class Solution {
    public int findPeakElement(int[] nums) {
        int low = 0, high = nums.length - 1;
        while(low < high) {
            int mid = low + (high - low) / 2;
```

```
            if(nums[mid] < nums[mid + 1]){
                low = mid + 1;
            }
            else high = mid;
        }
        return high;
    }
}
```

8) Search a 2D Matrix :

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        for(int i=0;i<matrix.length;i++){
            for(int j=0;j<matrix[0].length;j++){
                if(matrix[i][j] == target) return true;
            }
        }
        return false;
```

9) Koko Eating Bananas :

```
class Solution {
    public int minEatingSpeed(int[] piles, int h) {
        int n = piles.length;
        int max = 0;
        for(int i = 0; i < n; i++) {
            max = Math.max(max, piles[i]);
        }
        int l = 1, r = max;
        int res = r;
        while(l <= r) {
            int mid = l + (r - l) / 2;
            if(find(piles,h,mid)) {
                res = mid;
                r = mid - 1;
            }
            else l = mid + 1;
        }
        return res;
    }
    public boolean find(int[] piles, int h , int mid) {
        int sum = 0;
        for(int pile : piles) {
            sum += (pile + mid - 1) / mid;
        }
```

```
        return sum <= h;
    }
}
```

10) median of two sorted arrays striver :

```
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        int n1 = nums1.length;
        int n2 = nums2.length;
        int n = n1 + n2;
        int nums[] = new int[n];
        int i = 0; int j = 0; int k = 0;
        while(i < n1 && j < n2) {
            if(nums1[i] < nums2[j]){
                nums[k++] = nums1[i++];
            }
            else {
                nums[k++] = nums2[j++];
            }
        }
        while (i < n1) {
            nums[k++] = nums1[i++];
        }
        while (j < n2) {
            nums[k++] = nums2[j++];
        }
        if(n % 2 != 0) {
            return nums[n/2];
        }
        else {
            return (nums[(n/2) - 1] + nums[n/2]) / 2.0;
        }
    }
}
```

1) Running Sum of 1d Array :

```java
class Solution {
    public int[] runningSum(int[] nums) {
        int n = nums.length;

        int[] prefix = new int[n];
        prefix[0] = nums[0];
        int sum = nums[0];
        for(int i = 1; i < n; i++) {
            sum += nums[i];
            prefix[i] = sum;
        }
        return prefix;
    }
}
```

2) Range Sum Query - Immutable :

```java
class NumArray {
    private int[] prefix;
    public NumArray(int[] nums) {
        int n = nums.length;
        prefix = new int[n + 1];
        int sum = 0;
        for(int i = 0; i < n; i++) {
            sum += nums[i];
            prefix[i] = sum;
        }
    }
    public int sumRange(int left, int right) {
        return prefix[right + 1] - prefix[left];
    }
}
```

3) Find the Middle Index in Array :

```java
class Solution {
    public int findMiddleIndex(int[] nums) {
        int n = nums.length;
        int[] left = new int[n];
        int lsum = 0;
        for(int i = 1; i < n; i++) {
            lsum += nums[i - 1];
            left[i] = lsum;
        }
        int[] right = new int[n];
        int rsum = 0;
        for(int i = n - 2; i >= 0; i--) {
            rsum += nums[i + 1];
            right[i] = rsum;
        }

        for(int i = 0; i < n; i++) {
            if(left[i] == right[i]) return i;
        }
        return -1;
    }
}
```

4) Find Pivot Index :

```java
class Solution {
    public int pivotIndex(int[] nums) {
        int n = nums.length;
        int[] left = new int[n];
        int lsum = 0;
        for(int i = 1; i < n; i++) {
            lsum += nums[i - 1];
            left[i] = lsum;
        }
        int[] right = new int[n];
        int rsum = 0;
        for(int i = n - 2; i >= 0; i--) {
            rsum += nums[i + 1];
            right[i] = rsum;
        }

        for(int i = 0; i < n; i++) {
```

```
            if(left[i] == right[i]) return i;
        }
        return -1;
    }
}
```

5) Subarray Sum Equals K :

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        int sum = 0;
        HashMap<Integer, Integer> map = new HashMap<>();
        map.put(sum, 1);
        int count = 0;
        for(int ele : nums) {
            sum += ele;
            if(map.containsKey(sum - k)) {
                count += map.get(sum - k);
            }
            map.put(sum, map.getOrDefault(sum, 0) + 1);
        }
        return count;
    }
}
```

6) Contiguous Array :

```
class Solution {
    public int findMaxLength(int[] nums) {
        int n = nums.length;
        Map<Integer, Integer> mp = new HashMap<>();
        int sum = 0;
        int subArrayLength = 0;
        for (int i = 0; i < n; i++) {
            sum += nums[i] == 0 ? -1 : 1;
            if (sum == 0) {
                subArrayLength = i + 1;
            } else if (mp.containsKey(sum)) {
                subArrayLength = Math.max(subArrayLength, i - mp.get(sum));
            } else {
                mp.put(sum, i);
            }
        }
        return subArrayLength;
    }
```

```
}
```

7) Continuous Subarray Sum :

```
class Solution {
    public boolean checkSubarraySum(int[] nums, int k) {
       Map<Integer, Integer> remainderIndexMap = new HashMap<>();
        remainderIndexMap.put(0, -1);
        int sum = 0;
        for (int i = 0; i < nums.length; i++) {
            sum += nums[i];
            int remainder = sum % k;
            if (remainderIndexMap.containsKey(remainder)) {
                if (i - remainderIndexMap.get(remainder) > 1) {
                    return true;
                }
            } else {
                remainderIndexMap.put(remainder, i);
            }
        }
        return false;
    }
}
```